

Department of Computer Science

Faculty of Physical Sciences

Ahmadu Bello University, Zaria

COSC 211 : Object Oriented Programming I - LAB10

Objectives:

- Learn to think recursively
- Learn how to write simple recursive methods

1. Thinking Recursively

It is natural to think of task that involves repetition in terms of loops. For example that task of computing the factorial of a number n , could be thought of as finding cumulative product of numbers from 1 to n . That is using the formula”

$$n! = 1 * \dots * n \text{ or } 1 \text{ if } n = 0.$$

Another approach to solving problems that involves repetition is **divide-and-conquer** approach or **recursion**. In this approach, the solution is defined in terms of a smaller version of the problem. This simplification of the problem is repeated until the smallest form of the problems is reached for which the solution is obvious. For example, the factorial of n could be defined as:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n(n-1)! & \text{if } n > 0 \end{cases}$$

Some examples of problems that can be thought of in this way are described below:

$$\text{sumFromOneTo}(n) = \begin{cases} 0 & \text{if } n \leq 1 \\ \text{sumFromOneTo}(n-1) + n & \text{if } n > 1 \end{cases}$$

$$\text{arrayMinimum}(\text{array}, \text{start}) = \begin{cases} \text{array}[\text{start}] & \text{if } \text{start} = \text{length} - 1 \\ \min(\text{array}[\text{start}], \text{arrayMinimum}(\text{array}, \text{start} + 1)) & \text{if } \text{start} < \text{length} - 1 \end{cases}$$

$$\text{isMember}(\text{element}, \text{array}, \text{start}) = \begin{cases} \text{false} & \text{if } \text{start} \geq \text{length} \\ \text{true} & \text{if } \text{array}[\text{start}] == \text{element} \\ \text{isMember}(\text{element}, \text{array}, \text{start} + 1), & \text{if } \text{array}[\text{start}] \neq \text{element} \end{cases}$$

$$\text{printRange}(\text{start}, \text{stop}, \text{step}) = \begin{cases} \text{print nothing} & \text{if } \text{start} > \text{stop} \\ \text{print start; printRange}(\text{start} + \text{step}, \text{stop}, \text{step}) & \text{if } \text{start} \leq \text{stop} \end{cases}$$

2. Writing recursive methods in Java

One of the beauty of recursive algorithms is that they can be implemented almost directly in Java. The code is generally shorter than the loop version and it usually requires less number of variables and assignments. The following examples implements the algorithms described above.

Example 1: Computes the sum of integers from 1 to a given number, n.

Recursive	Iterative
<pre>import java.io.*; public class SumFromOneTo { public static void main(String args[]) { System.out.println(sumFromOneTo(10)); } public static int sumFromOneTo(int n) { if (n <= 0) return 0; else return sumFromOneTo(n-1)+n; } }</pre>	<pre>import java.io.*; public class IterativeSumFromOneTo { public static void main(String args[]) { System.out.println(sumFromOneTo(10)); } public static int sumFromOneTo(int n) { int sum=0; for (int i=1; i<=n; i++) sum+=i; return sum; } }</pre>

Example 2: Find the minimum element from an array.

Recursive
<pre>import java.util.Scanner; public class ArrayMinimum { public static void main(String[] args) { Scanner stdin = new Scanner(System.in); double[] number = new double[10]; for(int k = 0; k < number.length; k++) { System.out.print("Please enter element#" + (k + 1)+" ":); number[k] = stdin.nextDouble(); } System.out.println("The minimum value in the array is " + arrayMinimum(number, 0)); System.out.println(); } public static double arrayMinimum(double[] x, int start) { if (start == x.length-1) return x[x.length-1]; else</pre>

```

        return Math.min(x[start], arrayMinimum(x, start+1));
    }
}

```

Iterative

```

public static double arrayMinimum(double[] x) {
    double min = x[0];
    for (int i=1; i<x.length; i++)
        if (x[i] < min)
            min = x[i];
    return min;
}

```

Example 3: Check if an element is contained in an array.

Recursive

```

import java.util.Scanner;

public class Member {
    public static void main(String[] args) {
        Scanner stdin = new Scanner(System.in);

        double[] number = new double[10];
        for(int k = 0; k < number.length; k++) {
            System.out.print("Please enter element#" + (k + 1)+" : ");
            number[k] = stdin.nextDouble();
        }

        System.out.print("\nNow enter element to search for: ");
        double element = stdin.nextDouble();

        if (isMember(element, number, 0))
            System.out.println(element+ " is contained in the array");
        else
            System.out.println(element+ " is NOT contained in the array");
    }

    public static boolean isMember(double e, double[] x, int start) {
        if(start >= x.length)
            return false;
        else if (e == x[start])
            return true;
        else
            return isMember(e, x, start+1);
    }
}

```

Iterative

```

public static boolean isMember(double e, double[] x) {
    for (int i=0; i<x.length; i++)
        if (x[i] == e)

```

```

        return true;

    return false;

}

```

Example 4: Print numbers from a given starting value to a given stopping value incrementing using a given stepping value.

Recursive

```

import java.util.Scanner;

public class PrintRange {

    public static void main(String args[]) throws IOException {

        Scanner stdin = new Scanner(System.in);

        System.out.print("Enter starting value: ");

        int start = stdin.nextInt();

        System.out.print("Enter stoping value: ");

        int stop = stdin.nextInt();

        System.out.print("Enter stepping value: ");

        int step = stdin.nextInt();

        printRange(start, stop, step);

    }

    public static void printRange(int start, int stop, int step) {

        if (start <= stop) {

            System.out.println(start);

            printRange(start+step, stop, step);

        }

    }

}

```

Iterative

```

public static void printRange(int start, int stop, int step) {

    for (int i=start; i<stop; i+=step)

        System.out.println(i);

}

```

4. Assignments

- Write a recursive method that returns the sum of elements in an array of double. Test your method by writing a main method that creates an array of double of size 10, initialize it by reading data from the user and calling your method to compute the sum and print it. See fig 1 below:

```

MS-DOS Batch File D:\PROGRA~1\JCreator\GE2001.exe
Please enter element#1: 1
Please enter element#2: 4
Please enter element#3: 3
Please enter element#4: 2
Please enter element#5: 7
Please enter element#6: 6
Please enter element#7: 5
Please enter element#8: 7
Please enter element#9: 8
Please enter element#10: 9
The array sum is 52.0
Press any key to continue...

```

fig 1

```

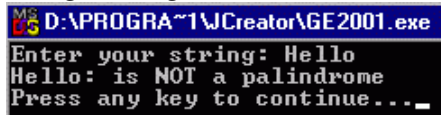
MS-DOS Batch File D:\PROGRA~1\JCreator\GE2001.exe
Please enter element#1: 1
Please enter element#2: 2
Please enter element#3: -5
Please enter element#4: -7
Please enter element#5: 2
Please enter element#6: -6
Please enter element#7: 9
Please enter element#8: 3
Please enter element#9: -5
Please enter element#10: 8
The array sum is 2.0
Number of positive elements is 6
Press any key to continue...

```

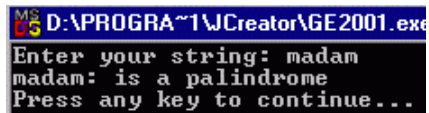
fig 2

- Improve your program in 1 above by writing another recursive method that counts the number of positive elements in the array. add a statement inside the main method to call this method so that you program prints both the sum and the number of positive elements in the array. See figure 2 above.

3. Write a recursive method that returns true if a string passed to it as parameter is a palindrome (palindrome is a string that reads the same both forward and backward). Test your method by writing a main method that reads a string from the user and then calls your method to know if the string is a palindrome and prints an appropriate message. See fig 3 below:

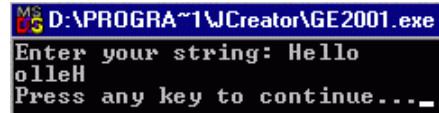


```
MS-DOS D:\PROGRAMS\1\Creator\GE2001.exe
Enter your string: Hello
Hello: is NOT a palindrome
Press any key to continue...
```



```
MS-DOS D:\PROGRAMS\1\Creator\GE2001.exe
Enter your string: madam
madam: is a palindrome
Press any key to continue...
```

fig 3



```
MS-DOS D:\PROGRAMS\1\Creator\GE2001.exe
Enter your string: Hello
olleH
Press any key to continue...
```

fig 4

4. Write a recursive method that prints a string passed to it as parameter in reverse order. Test your program by writing a main method that reads a string from the user and calls your method to print it in reverse order. See the fig 4. above.